

Arena Zero - developer guide

A 3D arena shooter for a phone held sideways: you against computer players ("bots") in a small walled arena, with no network and no waiting. It runs on the engine (the loop, the services, the layers; see `docs/GUIDE.md`) and, like the other games, adds only `src/game/`. This guide explains how it is built so that you could rebuild it, and where to turn the knobs.

1. The game

One match is a **Deathmatch** (free for all, first to 20 kills) or a **Team deathmatch** (you and your teammates against the other side, first to 40), with a five minute clock. You choose the mode, how clever the bots are (Easy, Normal, Hard), how many play (4, 6, 8 or 10 including you) and the map (a number: the same number is always the same arena).

You spawn with a **Pulse Pistol** and an **Arc Rifle**. The arena holds pick-ups that come back after a while: health, armor, ammunition, and three bigger guns.

Gun	Fires	Damage	Head	Magazine / spare	Reload	Reach	Notes
Pulse Pistol	tap	22	x2	12 / never runs out	1.1 s	90 m	accurate, the fallback
Arc Rifle	hold, 11 a second	12	x1.8	30 / 150	1.7 s	100 m	the workhorse; it blooms (spreads) the longer you hold
Scatter	tap	10 pellets of 9	x1.5	6 / 36	2.2 s	40 m	devastating close, little use far away
Railgun	tap	90	x1.6	4 / 20	2.4 s	250 m	zooms 3.2x when aimed; the only gun that is a scope
Rocket Launcher	tap	85 + splash 4 m	-	4 / 16	2.3 s	120 m	a rocket you can see coming; hurts you at half strength

You have 100 health and up to 100 armor, which soaks up 60% of each hit until it is gone. A shot that lands on the head does more; a hit on the legs does less. You walk at 5.2 m/s, run at 7.4, jump 1.1 m, and step up onto anything lower than 0.55 m without jumping. After you die you are back in three seconds, and safe for two.

Every number above lives in `src/game/sim/config.ts`; the tables here are copies, that file is the truth.

2. The idea everything rests on: a simulation that is only numbers

`src/game/sim/` is a pure function of a seed and the inputs people give it. It never reads a clock, never calls `Math.random`, never touches the DOM or Pixi or Three.js (the architecture test enforces this). The match advances in whole **ticks** of 1/60 s (`stepMatch(match, humanInput)`), and everything that happened is returned as a list of **events** (a shot, a hit, a death, a pick-up...) that the view, the sound and the HUD read; the rules never wait for any of them.

Why this matters: the bots can be tested (and tuned) by running a whole match in a few milliseconds; a bug is a failing test with a seed, not "it happened once on a phone"; the same match plays the same on any device.

`Input` is what a player (or a bot) wants: stick (`moveX`, `moveZ`), the way the head is turned (`yaw`, `pitch`, absolute: the controller owns the angles), and buttons. Looking is absolute so that the view can turn at the screen's speed while the simulation steps at 60 a second.

Angles. The Three.js camera's convention is used everywhere, so nothing is translated at the edge: yaw 0 looks along -z, **a positive yaw turns left**, a positive pitch looks up. Forward is (-sin yaw, -cos yaw), right is (cos yaw, -sin yaw). Everything that needs the other convention is a bug waiting.

3. The arena (`sim/arena.ts`)

The arena is a **heightfield**: 48 x 48 one-metre columns, each with a height. The outer wall is 6 m, crates are 0.5 or 1 m, steps and platforms go up to 2 m. That one choice makes most of the game cheap: walking is a height lookup, a bullet is a walk across a grid, the path finder is a grid search, and drawing is one instanced box per merged rectangle.

The generator places a mirrored layout (left-right and top-bottom, so both sides are fair), with random cover, then **proves** the result is playable: a flood fill from a spawn must reach every spawn, every pick-up spot and most of the floor, pockets that cannot be reached are filled solid, and a layout that fails is discarded for the next attempt (16 tries, then a plain fallback that always works). Twelve spawns (six a side) and the pick-up spots are laid out by the generator too.

4. Moving (`sim/physics.ts`)

A body is an upright cylinder (radius 0.35, 1.8 m tall). `heightUnder` is the tallest column the cylinder overlaps; moving is "try the x step, then the z step" so a body slides along a wall instead of stopping dead; the floor gives a ground height and gravity does the rest. On the ground the velocity closes on what the stick wants quickly (response 16 a second), in the air slowly (2.4), which is what makes jumping feel committed.

`raycastWorld` walks the grid cell by cell (a DDA) and returns the first wall, crate top or floor a ray meets; `rayVsCylinder` tests a ray against a body (it is how a bullet decides head, body or leg: by the height of the hit above the feet).

5. Shooting (`sim/weapons.ts`)

A shot is a ray with a random direction inside a cone. The cone (degrees) is the gun's own **spread**, plus **move spread** while you are moving, plus **bloom** that builds with every shot and is lost only once you let go of the trigger, plus a little more in the air; aiming down the sights narrows all of it. Damage falls off with distance between two ranges to a floor fraction (so a shotgun is hopeless at a distance), a head hit multiplies it, and armor soaks 60%. A rocket is a projectile with a speed that explodes where it lands: splash damage falls off by distance, shoves bodies away, and hurts the firer at half strength.

Holding the trigger repeats for every gun that is not a "tap" gun; a tap gun needs a fresh press (the simulation remembers whether the trigger was down last tick).

6. The bots (`sim/bots.ts`, `NavGrid` in `sim/arena.ts`)

A bot has the same body, the same guns and the same `Input` as you; it is not allowed to cheat, only to be patient. What makes it feel like a person is what it is *not* allowed to know and how slowly it does things:

- **Senses.** It sees what is in its field of view with a clear line (`lineOfSight`), and hears gunshots through `match.sounds` (who, where, when). It remembers where it last saw someone for a few seconds.
- **Reaction.** It does nothing about a new enemy until its reaction time has passed (0.6 s Easy, 0.34 Normal, 0.17 Hard).
- **Aim.** It turns at a limited speed (150, 270, 460 degrees a second) and aims with an error that wanders (6.5, 3.2, 1.4 degrees); it only fires when the crosshair is close enough that the shot could hit.
- **Moving.** It plans with A* over the heightfield (`NavGrid` : eight neighbours, no corner cutting, steps allowed up to the step height, then the path is pulled straight by line-of-sight tests), follows the waypoints, strafes while fighting, jumps what it must, and notices when it is stuck and picks a new goal.
- **Choices.** It picks a gun by distance, goes for health when hurt and a better gun when it is not, and wanders between spots when nothing is happening.

Measured over whole matches (the tests do this): accuracy is about 23%, 31% and 37%, a match lasts three to five minutes, and one bot's thinking costs about 0.02 ms a tick.

7. The rules of a match (`sim/match.ts`)

`createMatch({ seed, mode, difficulty, bots, human })` builds the arena and the actors (you are actor 0; `human: false` is a bots-only match, which is what plays behind the menu). `stepMatch` runs, in order: bring back anyone whose respawn time has come, move and shoot (in an order that **alternates** each tick: whoever acts first in a tick has the edge in a duel, so it is never the same actor), push bodies apart, move rockets, respawn pick-ups, check the end (the score limit, or the clock).

Spawn choice looks for a spawn point far from enemies and out of their sight, on your own side in a team match. A dead actor stays in the list, with the time it comes back.

8. Drawing (`src/game/view/`)

All of it reads the match and never changes it. One `world3d` owns a Three.js canvas laid *under* the engine's transparent 2D canvas (the engine guarantees its own canvas paints above; see the stacking test in `e2e/engine.spec.ts`), and draws a frame from a camera pose and the match.

- **The arena** (`arenaMesh.ts`): a floor plane and *one* `InstancedMesh` of unit boxes, one instance per merged rectangle. The material is a Lambert material with a small shader patch (`onBeforeCompile`) that looks the panel texture up by **world position on three axes**, so a one-metre crate and a long wall both get panels of the same size with no texture coordinates; the bright marks of the texture glow, the lower metre darkens like soot. The sky is a dome of vertex colours that follows the camera; fog fades the far arena.
- **The textures** (`textures.ts`) are drawn in code on a canvas: there are no image files in the game.
- **The people** (`characters.ts` , `skeleton.ts` , `ragdoll.ts`): see below.
- **Effects** (`effects.ts`): tracers, sparks, billboard flashes and smoke, floor rings, bullet-hole decals, explosions; each kind is a pool of instances, so a firefight is a handful of draw calls.
- **Pick-ups** (`pickups.ts`) spin and bob; **rockets** are stretched boxes in `world3d.ts` .
- **Your gun** (`viewModel.ts`) is drawn in its own little scene on top (after `clearDepth`), so it never pokes into a wall; it bobs as you walk, lags as you turn, kicks when it fires, drops for a reload, tilts as you run and slides to the middle when you aim.
- **Resolution** is a number the flow can lower (`setQuality`); see section 10.

The people: a skeleton, then a ragdoll

A body is **sixteen joints** (`skeleton.ts` : pelvis, spine, neck, head, two shoulders, elbows, hands, hips, knees and ankles). While someone is alive the joints come from `poseAlive` : a walk cycle (the swing and knee fold follow the stride the simulation counts), the trunk leaning into a run and back when the head looks up, and the arms reaching to hold the gun where the head looks, through a **two-bone IK** (`reach` : the elbow goes where the law of cosines says, bent toward a pole direction). `characters.ts` draws the pose with four instanced meshes: cylinders for limbs, spheres for joints and the helmet, boxes for the armor, the pack, the feet and the gun, and a glowing visor in the team's colour.

When someone dies, **the same joints are handed to a ragdoll** (`ragdoll.ts`): a Verlet simulation (each joint remembers where it was, so its velocity is the difference) with a distance link for every bone (and braces that keep the trunk a trunk, and limits so a knee or an elbow cannot fold flat), gravity, damping, collision of each joint (a small sphere) against the same heightfield, friction on whatever it touches, and a sleep once it has hardly moved. It starts with the velocity the body had, then **the killing shot throws it**: the view remembers the last lethal `hit` for each victim (weapon, direction, head or not) and shoves the joints near where it struck, harder for a shotgun or a railgun, with a second shove at an ankle so the body spins; an **explosion** throws every joint away from its centre, harder when nearer. The ragdoll lives only in the view: it is a picture of a body falling, and the rules never look at it. After a few seconds it shrinks away.

Your own death uses the same ragdoll, and the camera rides its head (`camera.ts` `deadView`) and turns to look at whoever did it.

9. Controls and the HUD

Phones first

The target is a phone held sideways, so the touch layout is the primary one and everything is checked at phone sizes (667 x 375 up to 932 x 430) with real multi-touch events:

- **Left thumb: a floating stick.** It appears where the thumb lands in the left half; pushing it all the way out runs. The stick's radius is a thumb's reach, not a fraction of a big screen (`stickRadius` , pure and tested).
- **Right thumb: look.** Dragging anywhere in the right half turns the view. The big **Fire** button *also* turns the view while it is held, so one thumb can shoot and aim.
- **Buttons:** Jump, Aim (a toggle), Reload, Next weapon, a bar of weapon slots, Pause, Scoreboard. Every one is at least 44 CSS pixels (the e2e test measures them at every phone size and checks nothing overlaps).
- **Left-handed** mirrors the layout.
- **Aim assist** (`aim.ts` , pure and tested): when an enemy you can see is within about 9 degrees of the crosshair, your turns shrink (up to half) and, while you are turning or firing, the view drifts toward them (never past them). **Auto fire** shoots while the crosshair is on an enemy (a ray test). Both can be switched off in Settings.
- **Look speed** and **field of view** are sliders; looking is scaled down while zoomed in.

On a desktop, W A S D, Shift, Space, R, 1-5, the wheel, Tab and Esc work, the mouse is captured (pointer lock) by a click, and letting go of it (Esc) pauses. If the browser will not capture the mouse, dragging with the right button looks instead. Whichever device you used last decides which help text is shown.

The HUD

Health and armor bars, the score and clock (your team and theirs, or you and the leader), a **radar** that turns with you (drawn from the arena's heights; enemies show only while you can see them or just after they fire), the kill feed, hit markers (white, yellow for a head, red for a kill), red arcs toward whoever shot you, a red edge when health is low, the crosshair (its gap is the actual spread, red when it is on an enemy), the ammunition and the bar of guns, toasts, and the banners (the countdown, "ELIMINATED... back in 3", the result). The HUD is `pointer-events: none` everywhere except its buttons, so the touch areas underneath get every other touch (a test checks it).

`ui/` is plain DOM: it is given plain values (`HudInfo` , `RadarInfo` ...) and reports presses through callbacks; it never sees the match. `touchpad.ts` turns touches into three things (a stick vector, how far a finger moved to look, button down/up).

10. The flow (`index.ts`) and the platform

`index.ts` runs the phases (menu with a bots-only match behind it, countdown, playing, paused, result), builds the `Input` each tick, handles every event (sound, vibration, the HUD, the ragdolls), and renders. It also:

- keeps the screen awake while you play (`wakeLock`), asks a phone for **fullscreen** and a **landscape lock** on the press of Play (the press is the gesture the browser needs), pauses when the window is hidden or the phone is turned upright, and shows a "turn your phone sideways" screen in portrait;
- measures its own frame rate and, if a device cannot keep up, draws **fewer pixels** (`quality.ts` : it only ever lowers, in steps, after waiting for the average to settle; Settings can pin it to Fast or Sharp);
- runs the **ads**: a rewarded ad on the death banner brings you back at once with full armor and the railgun and rocket launcher (once a match); an interstitial may play between matches (never after your first match, never after a short one).

Everything that touches a platform (haptics, audio, ads, storage) comes from the engine; none of it can throw into game code.

11. Tests, and what is not verified

`npm run check` (typecheck + unit tests) covers the arena generator (including that every layout is playable), movement and the rays, the weapons (spread, bloom, falloff, armor, rockets), the bots (they see, wait, aim, shoot, path, get unstuck, and accuracy goes up with the level), the match rules, the aim assist, the camera, the profile, the ragdoll (it falls, rests, keeps its bones a length, is thrown the way it was shot, stops at walls and lies on crates) and the architecture rules.

`npm run e2e` drives the real game in a phone-sized browser with real multi-touch events: the menu, a match from the countdown, every touch control, shooting a bot dead and seeing its body fall, dying and coming back, the rewarded ad, pausing, the scoreboard, the result, left-handed play, every control measured at seven screen sizes, an upright phone, and the desktop keyboard and mouse. It writes screenshots to `e2e/screenshots/`; **open them**: passing tests do not prove the canvas drew.

Not verified: the feel on a real phone (the test browser draws in software, slowly), real GPUs and frame rates, the Android and iOS builds, real ads, real vibration and sound output, and fullscreen/landscape-lock behaviour on real devices. The adaptive resolution and the numbers in `sim/config.ts` are the first things to tune after a day on a phone.

12. How to change things

To change	Edit
Any gun, health, speeds, bot levels, modes, the arena size	<code>sim/config.ts</code> (numbers only; the tests say what you broke)
The look of the arena (colours, glow, fog)	<code>view/palette.ts</code> , <code>view/textures.ts</code> , <code>view/arenaMesh.ts</code> , <code>view/world3d.ts</code>
How people look or fall	<code>view/characters.ts</code> (the parts), <code>view/skeleton.ts</code> (the pose), <code>view/ragdoll.ts</code> (gravity, damping, friction)
How the touch controls feel	<code>controls.ts</code> (<code>TOUCH_TURN</code> , <code>MOUSE_TURN</code>), <code>aim.ts</code> , <code>ui/touchpad.ts</code> (<code>stickRadius</code>), <code>ui/styles.css</code>
The HUD	<code>ui/ui.ts</code> , <code>ui/styles.css</code> , and <code>refreshHud</code> in <code>index.ts</code>
Sounds	<code>sfx.ts</code>

To change	Edit
A new map rule or pick-up	<code>sim/arena.ts</code> (the generator and its proof), <code>sim/match.ts</code>

URL flags (development builds): `?seed=N`, `?mode=ffa|tdm`, `?level=easy|normal|hard`, `?bots=3|5|7|9`, `?autostart=1`, `?countdown=0`, `?quality=0.5..1`, and the engine's `?ads=no-fill|skip`.

`window.__game.debug` has the live match and a few levers for setting up a scene (teleport, put an actor somewhere, give a gun, kill someone, face an actor).

13. Ship

`npm run build` makes `dist/`; `npm run android:setup` then `npm run android` build the Android app (the native project is generated and never edited by hand; `scripts/setup-native.mjs` locks it to landscape). The same build is an installable web app (`public/manifest.webmanifest`, `"orientation": "landscape"`). The ad IDs are test IDs until `VITE_ADS_TESTING=false` and the real ones are set (see the engine guide).